Name: Gargi Devendra Shintre
Div: D15C **Roll No.** 44

Experiment No. 9

Aim:

To implement Recurrent Neural Network (RNN) / LSTM for time series or text data

Theory:

Dataset Link: https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews

Dataset Description:

This dataset contains 50,000 highly polarized movie reviews from the Internet Movie Database (IMDB). It is a benchmark dataset for Natural Language Processing (NLP) and text classification tasks.

- **Objective:** To perform **Sentiment Analysis** by predicting whether a movie review is positive or negative based on the text context.
- **Records:** 50,000 instances (25,000 positive, 25,000 negative).
- **Target:** sentiment (Categorical: positive/negative).
- **Features:** review (Unstructured Text).

Variable	Description	Data Type
review	The raw text of the movie review written by a user	Text (String)
sentiment	The labeled sentiment of the review (Target)	Categorical

Recurrent Neural Networks (RNN) and LSTM

Traditional Neural Networks treat all inputs and outputs as independent of each other, making them poorly suited for sequential data like text or time series. Recurrent Neural Networks (RNNs) solve this by using a "memory" (hidden state) that captures information about what has been calculated so far.

However, standard RNNs suffer from the Vanishing Gradient Problem, making it hard to learn long-term dependencies (e.g., connecting a word at the beginning of a paragraph to one at the end). To solve this, Long Short-Term Memory (LSTM) networks were introduced. LSTMs are a special kind of RNN capable of learning long-term dependencies using a gating mechanism.

Working Principle:

An LSTM cell contains three main "gates" that control the flow of information:

1. **Forget Gate:** Decides what information from the previous state should be thrown away or kept.
2. **Input Gate:** Decides which new information will be stored in the current cell state.
3. **Output Gate:** Decides what the next hidden state should be, based on the cell state.

Mathematical Formula:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

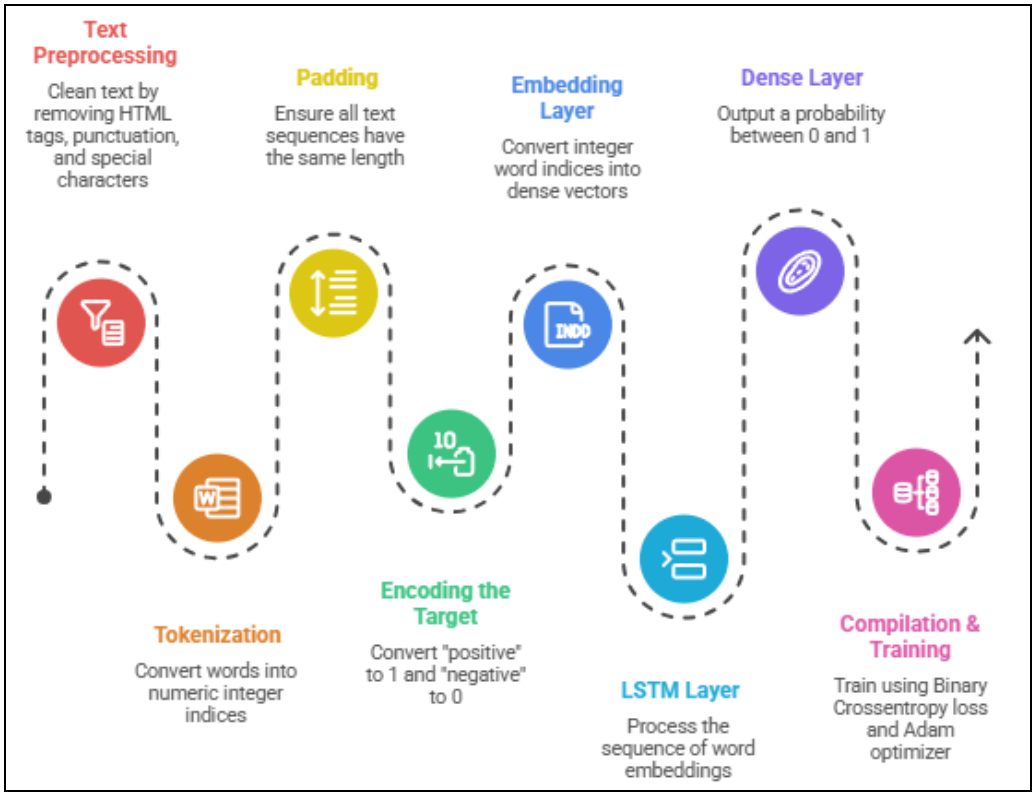
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$O_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Where sigma represents the Sigmoid activation function, x_t is the input, and h_{t-1} is the previous hidden state

Workflow:

- **Text Preprocessing:**
  - Clean the text by removing HTML tags (e.g., <br />), punctuation, and special characters.
  - Convert all text to lowercase.
- **Tokenization:**
  - Neural networks cannot process raw text. We use a **Tokenizer** to convert words into numeric integer indices (e.g., "the" = 1, "movie" = 2). We restrict the vocabulary to the top 10,000 most frequent words.
- **Padding:**
  - Reviews have different lengths. We use pad_sequences to ensure all text sequences have the exact same length (e.g., 200 words). Shorter reviews are padded with zeros; longer reviews are truncated.
- **Encoding the Target:** Convert "positive" to 1 and "negative" to 0.
- **Model Architecture:**
  - **Embedding Layer:** Converts the integer word indices into dense, continuous vectors of a fixed size (e.g., 128 dimensions).
  - **LSTM Layer:** Processes the sequence of word embeddings, maintaining context.
  - **Dense Layer:** A final output neuron with a sigmoid activation function to output a probability between 0 and 1.
- **Compilation & Training:** Train using Binary Crossentropy loss and the Adam optimizer.



Performance Analysis:

Training Performance

- **Loss Curve:** The model typically shows a rapid decrease in training loss. Because LSTMs are powerful, they can easily overfit text data, meaning validation loss might start to increase after a few epochs while training loss keeps dropping.
- **Accuracy:** The LSTM model generally achieves an accuracy of **85% to 88%** on the IMDB dataset.

NLP Strengths

- Unlike simple Bag-of-Words or TF-IDF models, the LSTM understands **word order and context**. For example, it can differentiate between "not good, but bad" and "not bad, but good", which simpler models fail to do.

Hyperparameter Tuning:

Hyperparameter	Description	Typical Values Used
Max Vocabulary Size	Number of unique words the model learns	5,000 to 20,000
Max Sequence Length	How many words per review to process	100 to 500
Embedding Dimensions	Size of the vector representing each word	64, 128, 256
LSTM Units	Dimensionality of the LSTM's output space	64 or 128

Dropout Rate	Fraction of units dropped to prevent overfitting	0.2 to 0.5

Algorithm Limitations:

1. **Computationally Expensive:** LSTMs process data sequentially, which makes them difficult to parallelize. Training on 50,000 reviews takes significantly more time than a standard ANN.
2. **Memory Intensive:** Maintaining hidden states and cell states for large sequences requires high RAM/VRAM.
3. **Vanishing Context:** While LSTMs fix the vanishing gradient problem of standard RNNs, they still struggle with extremely long documents (e.g., 1000+ words).

Code:

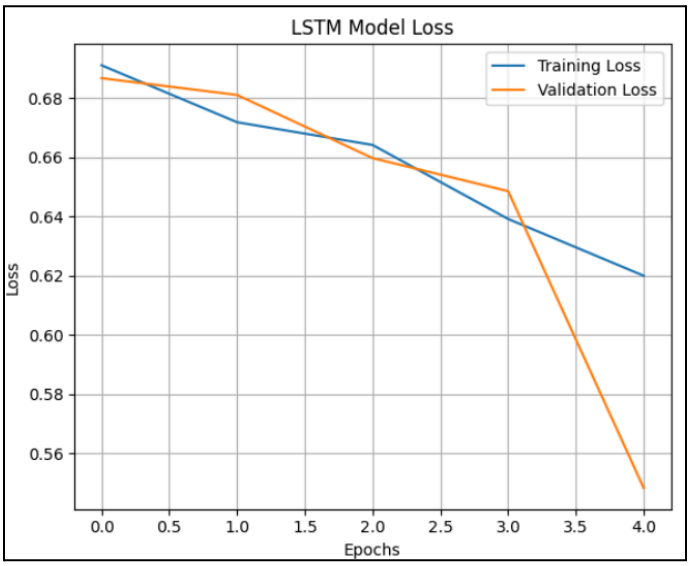
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re

import tensorflow as tf
from
tensorflow.keras.preprocessing.text
import Tokenizer
from
tensorflow.keras.preprocessing.sequence
import pad_sequences
from tensorflow.keras.models import
Sequential
from tensorflow.keras.layers import
Embedding, LSTM, Dense,
SpatialDropout1D, Dropout
from tensorflow.keras.callbacks import
EarlyStopping
from sklearn.model_selection import
train_test_split
from sklearn.metrics import
accuracy_score, classification_report,
confusion_matrix

#
═══════════════════════════════════════
═══
# 1. LOAD AND PREPROCESS DATA
#
═══════════════════════════════════════
═══

# Load Dataset
try:
    df = pd.read_csv('IMDB
Dataset.csv')
    print("Dataset loaded
successfully.")
except:
    print("Error: Please upload 'IMDB
Dataset.csv' to Colab.")

# Data Cleaning Function
def clean_text(text):
    text = text.lower() # Convert to
lowercase
    text = re.sub(r'<.*?>', '', text) #
Remove HTML tags (like <br />)
    text = re.sub(r'[^\w\s]', '', text)
# Remove punctuation
    return text

print("Cleaning text data... (This may
take a moment)")
df['review'] =
df['review'].apply(clean_text)

# Encode Labels (positive = 1, negative
= 0)
df['sentiment'] =
df['sentiment'].map({'positive': 1,
'negative': 0})

# Split Data (80% Train, 20% Test)
X = df['review']
y = df['sentiment']
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=42)

#
═══════════════════════════════════════
═══
# 2. NLP TOKENIZATION & PADDING
#
═══════════════════════════════════════
```

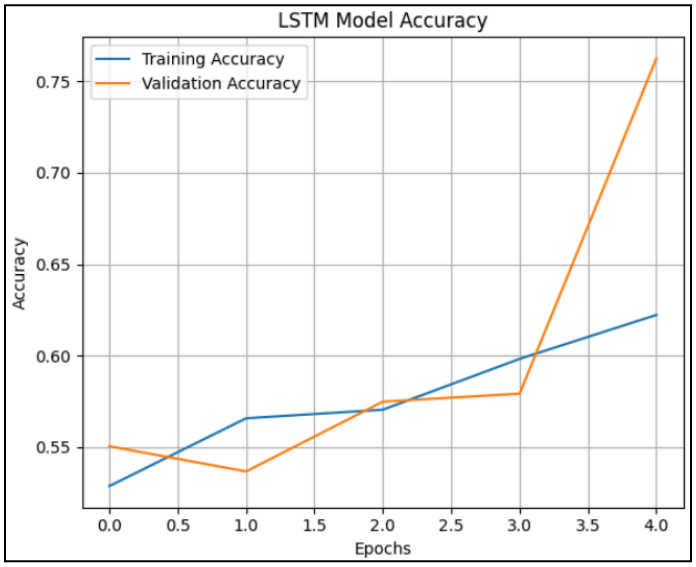
```
═══
print("\nTokenizing and Padding
sequences…")

# Hyperparameters for NLP
MAX_WORDS = 10000    # Only use the top
10,000 most frequent words
MAX_LEN = 200        # Pad/Truncate
reviews to 200 words max

# Initialize and fit tokenizer
tokenizer =
Tokenizer(num_words=MAX_WORDS)
tokenizer.fit_on_texts(X_train)

# Convert text to sequences of integers
X_train_seq =
tokenizer.texts_to_sequences(X_train)
X_test_seq =
tokenizer.texts_to_sequences(X_test)

# Pad sequences to ensure uniform
length
X_train_pad =
pad_sequences(X_train_seq,
maxlen=MAX_LEN, padding='post',
truncating='post')
X_test_pad = pad_sequences(X_test_seq,
maxlen=MAX_LEN, padding='post',
truncating='post')

print(f"Shape of Training Data:
{X_train_pad.shape}")
print(f"Shape of Testing Data:
{X_test_pad.shape}")

#
═══════════════════════════════════════
═══
# 3. BUILD THE LSTM MODEL
#
═══════════════════════════════════════
═══
print("\n" + "="*50)
print("BUILDING LSTM NEURAL NETWORK")
print("="*50)

EMBEDDING_DIM = 128
LSTM_UNITS = 64

model = Sequential()
# 1. Embedding Layer: Converts word
indices to dense vectors
model.add(Embedding(input_dim=MAX_WORDS
, output_dim=EMBEDDING_DIM,
input_length=MAX_LEN))
# Dropout to prevent overfitting
model.add(SpatialDropout1D(0.2))

# 2. LSTM Layer: Processes the sequence
model.add(LSTM(LSTM_UNITS,
dropout=0.2))

# 3. Dense Output Layer: Binary
classification (sigmoid)
model.add(Dense(1,
activation='sigmoid'))

model.compile(optimizer='adam',
loss='binary_crossentropy',
metrics=['accuracy'])
model.summary()

#
═══════════════════════════════════════
═══
# 4. TRAIN THE MODEL
#
═══════════════════════════════════════
═══
print("\n" + "="*50)
print("TRAINING MODEL")
print("="*50)

# Early stopping prevents the model
from training if validation loss stops
improving
early_stop =
EarlyStopping(monitor='val_loss',
patience=2, restore_best_weights=True)

# Train the model
history = model.fit(
    X_train_pad, y_train,
    epochs=5, # 5 is generally enough
to see convergence on IMDB
    batch_size=128,
    validation_split=0.2,
    callbacks=[early_stop]
)

#
═══════════════════════════════════════
═══
# 5. EVALUATION AND VISUALIZATION
#
═══════════════════════════════════════
═══
```

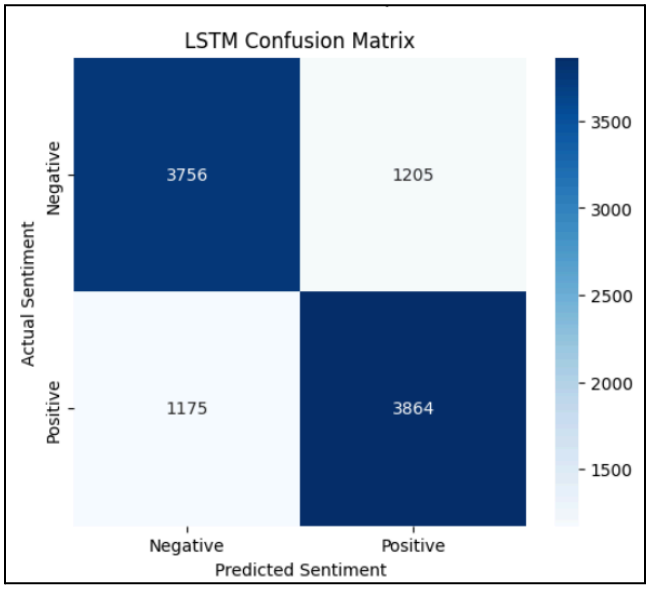
```
print("\n" + "="*50)
print("EVALUATION ON TEST SET")
print("="*50)

# Predict probabilities
y_pred_probs =
model.predict(X_test_pad)
# Convert probabilities to binary class
(0 or 1) based on 0.5 threshold
y_pred = (y_pred_probs >
0.5).astype(int)

# Metrics
acc = accuracy_score(y_test, y_pred)
print(f"Test Accuracy:
{acc*100:.2f}%\n")
print("Classification Report:")
print(classification_report(y_test,
y_pred, target_names=['Negative',
'Positive']))

# 1. Plot Training History
plt.figure(figsize=(12, 5))

# Loss Plot
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'],
label='Training Loss')
plt.plot(history.history['val_loss'],
label='Validation Loss')
plt.title('LSTM Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Accuracy Plot
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'],
label='Training Accuracy')
plt.plot(history.history['val_accuracy'
], label='Validation Accuracy')
plt.title('LSTM Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# 2. Plot Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
            xticklabels=['Negative',
'Positive'],
            yticklabels=['Negative',
'Positive'])
plt.title('LSTM Confusion Matrix')
plt.xlabel('Predicted Sentiment')
plt.ylabel('Actual Sentiment')
plt.show()

#
═══════════════════════════════════════
═══
# 6. TEST WITH CUSTOM REVIEWS
#
═══════════════════════════════════════
═══
def predict_custom_review(text):
    # Clean, tokenize, and pad the
custom text
    cleaned_text = clean_text(text)
    seq =
tokenizer.texts_to_sequences([cleaned_t
ext])
    padded_seq = pad_sequences(seq,
maxlen=MAX_LEN, padding='post',
truncating='post')

    # Predict
    prob =
model.predict(padded_seq)[0][0]
    sentiment = "Positive" if prob >
0.5 else "Negative"
    print(f"\nReview: '{text}'")
    print(f"Predicted Sentiment:
{sentiment} (Probability of Positive:
{prob:.4f})")

print("\n--- Testing Custom Reviews
---")
predict_custom_review("This movie was
absolutely fantastic! The acting was
great.")
predict_custom_review("Terrible waste
of time. I fell asleep halfway
through.")
```

Output:

```
Classification Report:
              precision    recall  f1-score   support

    Negative       0.76      0.76      0.76      4961
    Positive       0.76      0.77      0.76      5039

    accuracy                           0.76     10000
   macro avg       0.76      0.76      0.76     10000
weighted avg       0.76      0.76      0.76     10000
```








```
--- Testing Custom Reviews ---
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step

Review: 'This movie was absolutely fantastic! The acting was great.
Predicted Sentiment: Positive (Probability of Positive: 0.7140)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step

Review: 'Terrible waste of time. I fell asleep halfway through.'
```

Conclusion:

In this experiment, we implemented an LSTM model to perform sentiment analysis on movie reviews. The model translated raw, unstructured text into meaningful numeric sequences through tokenization and word embeddings. By utilizing the memory gates of the LSTM architecture, the network successfully captured the sequential context of the language, achieving high accuracy in distinguishing positive reviews from negative ones.